DMC | INSTITUTE

Diploma Ingeniería de Software con IA

Sesión 1: Repaso sobre diseño frontend engineering

Docente: Jorge Lozano

DMC | INSTITUTE

Agenda de la sesión

1	**Apertura · Objetivos, ruta del curso y proyecto integrador** Qué construiremos en las 5 sesiones: TurnoYa	8 min
2	**Principios de UX/UI: fundamentos, heurísticas y leyes** Jerarquía, tipografía, color, Gestalt, 10 heurísticas de Nielsen y 6 leyes de UX	27 min
3	**Accesibilidad web: WCAG 2.2** POUR, niveles A/AA/AAA, criterios clave, contraste, ARIA y cómo probar	20 min
4	**Taller 1 · Auditoría UX/UI y corrección accesible** Auditas la pantalla de reserva y corriges 4 grupos de problemas en código	25 min
5	**Receso** Pausa activa	10 min
6	**Components & Design Systems + Taller 2** Atomic design, tokens en 3 capas, anatomía y estados · Tokens y componentes	35 min
7	**Responsive & Mobile Design** Mobile-first, unidades, grid intrínseco, container queries · Catálogo responsive	25 min
8	**Cierre · Recursos y próxima sesión** Resumen y preguntas	5 min

PROYECTO INTEGRADOR

TurnoYa: reservas de citas para negocios de servicios

EL PROBLEMA

Barberías, consultorios y spas agendan por WhatsApp y llamadas: **dobles reservas**, clientes que **no llegan** y el dueño contestando mensajes en vez de atender.

Dos usuarios, dos contextos

- **Camila, 29 · cliente**
  Reserva desde el celular, en el bus, con una mano. No quiere crear cuenta. Necesita ver horarios libres y confirmar en menos de 60 segundos.

- **Don Raúl, 52 · dueño**
  Revisa la agenda en una tablet en el mostrador, entre cliente y cliente. Necesita ver el día de un vistazo y bloquear horarios.

Pantallas del MVP

Catálogo de servicios
HU-01 · móvil

Selección de horario
HU-02 · móvil

Reserva y confirmación
HU-03 · móvil

Agenda del negocio
HU-05/06 · tablet · tarea

01

Principios de UX/UI

Fundamentos visuales, heurísticas de usabilidad y leyes de UX que todo frontend debe dominar

UX y UI: dos disciplinas, un mismo objetivo

UX (User Experience) es cómo se siente y funciona el producto para la persona: si logra su objetivo, con cuánto esfuerzo y con qué confianza. La norma ISO 9241-210 la define como las percepciones y respuestas de una persona que resultan del uso de un producto. **UI (User Interface)** es la capa visible e interactiva donde esa experiencia ocurre.

UX responde: ¿funciona?

- Flujos y número de pasos
- Arquitectura de información
- Estados: vacío, carga, error, éxito
- Microcopy: qué dice cada texto
- Métricas: tasa de éxito, tiempo, abandono

UI responde: ¿se entiende?

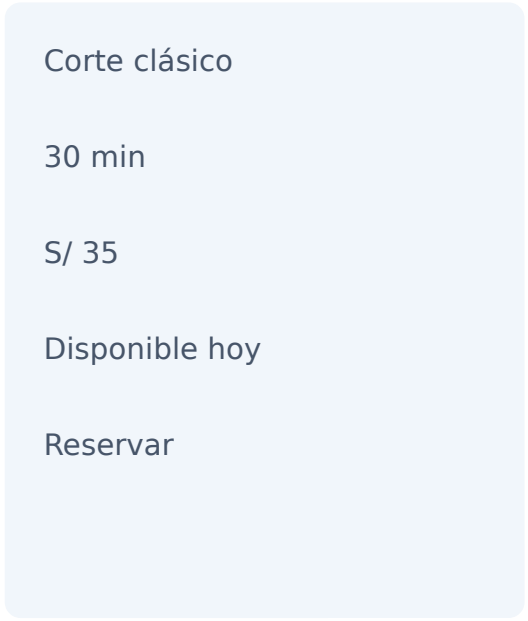
- Jerarquía visual
- Tipografía, color y espaciado
- Componentes y sus estados
- Consistencia entre pantallas
- Feedback visual de cada acción

Jerarquía visual: decidir qué se ve primero

La jerarquía visual ordena la atención: el ojo recorre la pantalla de lo más llamativo a lo menos. Si todo destaca, nada destaca. Se construye con **5 herramientas** que se combinan.

- **Tamaño**
  Lo más grande se lee primero. Usa una escala con saltos claros (16 → 20 → 24 → 32 px), no 16 vs 17 px.
- **Peso**
  Negrita (600–700) para títulos y datos clave; regular (400) para el cuerpo.
- **Color y contraste**
  El color de marca solo en lo accionable o importante. El texto secundario en gris.
- **Posición**
  En occidente se lee de arriba a la izquierda hacia abajo a la derecha.
- **Espacio**
  El espacio en blanco aísla y da importancia. Agrupa lo relacionado y separa lo distinto.

Mismo contenido, dos jerarquías



✗ **todo pesa igual**



✓ **nombre → precio → acción**

Tipografía: legibilidad antes que estilo

En una interfaz el 90 % del contenido es texto. Una buena tipografía de UI se decide con **números**, no con gusto: escala, interlineado, longitud de línea y unidades.

- **Escala modular**
  Multiplica una base por una razón fija. Base 16 px × 1.25 → 16, 20, 25, 31… Así los saltos se perciben como intencionales.

- **Interlineado (line-height)**
  1.5 para párrafos, 1.2 para títulos. Sin unidades: `line-height: 1.5` escala con la letra.

- **Longitud de línea**
  Entre 45 y 75 caracteres por línea. Más larga cansa; más corta corta la lectura. En CSS: `max-width: 65ch`.

- **Pocas familias**
  Una o dos familias como máximo. Las fuentes del sistema (`system-ui`) cargan al instante.

- **Unidades rem**
  `1rem` = tamaño base que eligió el usuario en su navegador. Con `px` ignoras esa preferencia.

Escala tipográfica

```
:root {
  /* base 16px, razón 1.25 (tercera mayor) */
  --font-size-sm:  .875rem;  /* 14px · ayudas   */
  --font-size-md:  1rem;     /* 16px · cuerpo   */
  --font-size-lg:  1.25rem;  /* 20px · subtítulo */
  --font-size-xl:  1.5rem;   /* 24px · título    */
  --font-size-2xl: 2rem;     /* 32px · portada   */
}
body {
  font-family: system-ui, "Segoe UI", sans-serif;
  font-size: var(--font-size-md);
  line-height: 1.5;
}
h1, h2 { line-height: 1.2; }
p      { max-width: 65ch; }   /* 45–75 caracteres */
```

Color: roles, proporciones y significado

En UI el color no es decoración: **comunica jerarquía y estado**. Por eso se organiza por **roles**, no por tonos favoritos.

Marca / acción
Botones primarios, links, selección

Texto
Títulos y cuerpo

Texto secundario
Ayudas, metadatos

Fondo / superficie
Canvas, tarjetas

Error / peligro
Validación, acciones destructivas

Éxito
Confirmaciones, disponible

Regla 60-30-10 · 60 % neutro dominante, 30 % color secundario, 10 % acento para lo accionable.

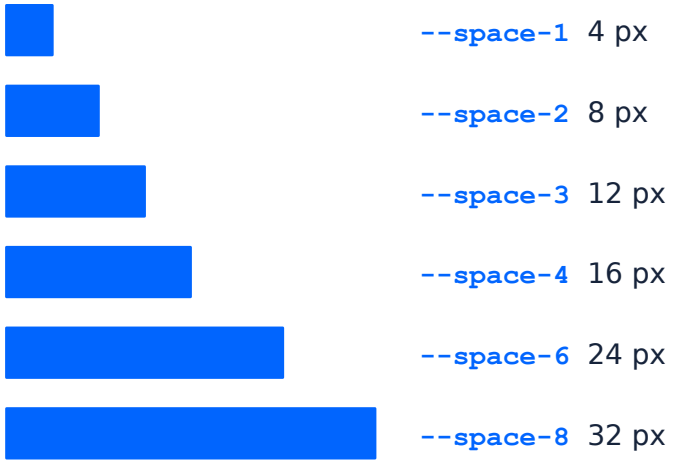
- **El color nunca va solo**
  Un estado (error, ocupado, seleccionado) siempre se acompaña de texto, ícono o forma. Así lo entiende quien no distingue colores (WCAG 1.4.1).
- **Un color, un significado**
  Si el teal es "acción", no lo uses para decorar un título. Si el rojo es "error", no lo uses para una oferta.
- **Contraste medible**
  Texto 4.5:1 y componentes 3:1. Lo calcularemos en el bloque de WCAG.
- **Modo oscuro ≠ invertir**
  Se redefinen los roles: superficies oscuras, marca más clara, sombras casi invisibles.

Espaciado y grilla: el ritmo invisible

El espaciado consistente es lo que hace que una interfaz se vea "profesional" sin que nadie sepa explicar por qué. La técnica más usada es la **grilla de 8 puntos** (con medio paso de 4).

- **Escala fija**
  Todos los márgenes, paddings y gaps salen de 4, 8, 12, 16, 24, 32, 40…
  Nunca 13 ni 17 px.
- **Proximidad = relación**
  Lo que está cerca se percibe como un grupo. El label va pegado a su campo (8 px) y lejos del campo anterior (20 px).
- **Interno ≤ externo**
  El espacio dentro de un componente debe ser menor o igual que el espacio que lo separa de otros.
- **Áreas táctiles**
  Mínimo 44 px de alto en móvil, con al menos 8 px entre objetivos.

Escala de espaciado de TurnoYa



Principios de Gestalt: cómo agrupa el cerebro

La psicología de la Gestalt (Alemania, década de 1920) explica que percibimos **conjuntos** antes que elementos sueltos. En UI se usa para que la estructura se entienda sin leer.

Proximidad

Elementos cercanos se ven como grupo.
En UI: Label pegado a su input; tarjetas separadas por 16 px.

Similitud

Lo que se parece se percibe como de la misma clase.
En UI: Todos los links del mismo color; todos los chips con la misma forma.

Región común

Lo que comparte un fondo o borde forma un grupo.
En UI: La tarjeta del servicio agrupa nombre, precio y botón.

Continuidad

El ojo sigue líneas y alineaciones.
En UI: Indicador de pasos 1 → 2 → 3; campos alineados a la izquierda.

Cierre

Completamos formas incompletas.
En UI: Un carrusel con la última tarjeta cortada invita a deslizar.

Figura-fondo

Separamos lo que está delante de lo que está atrás.
En UI: Un modal con fondo oscurecido enfoca la atención.

Las 10 heurísticas de Nielsen: qué son y para qué sirven

Son **10 principios generales de usabilidad** publicados por **Jakob Nielsen en 1994** (a partir de su trabajo con Rolf Molich en 1990). Se llaman heurísticas porque son reglas prácticas, no normas exactas. Con ellas se hace una **evaluación heurística**: revisar una interfaz contra cada principio sin necesidad de usuarios.

H1	**Visibilidad del estado del sistema**
H2	**Coincidencia con el mundo real**
H3	**Control y libertad del usuario**
H4	**Consistencia y estándares**
H5	**Prevención de errores**
H6	**Reconocer antes que recordar**
H7	**Flexibilidad y eficiencia de uso**
H8	**Diseño estético y minimalista**
H9	**Reconocer, diagnosticar y recuperarse de errores**
H10	**Ayuda y documentación**

Escala de severidad (Nielsen)

0	No es un problema
1	Cosmético: arreglar si sobra tiempo
2	Menor: prioridad baja
3	Mayor: prioridad alta
4	Catástrofe: impide la tarea, corregir ya

Según Nielsen, **3 a 5 evaluadores** encuentran cerca del **75 %** de los problemas. Uno solo encuentra alrededor de un tercio.

H1 · Visibilidad del estado del sistema

El sistema debe **mantener informado al usuario** sobre lo que está pasando, con un feedback adecuado y en un tiempo razonable. Nadie debería preguntarse "¿funcionó?".

En la práctica

- **Feedback inmediato**
  Cada clic produce un cambio visible en menos de 0.1 s.
- **Progreso visible**
  Indicador "Paso 2 de 3", barras de carga, contador de caracteres.
- **Estado actual claro**
  Qué filtro está activo, qué horario está elegido, en qué pantalla estoy.

Estado de carga accesible

```
btn.disabled = true;                // evita el doble envío
btn.setAttribute('aria-busy', 'true');
btn.textContent = 'Reservando…';
<p aria-live="polite">Paso 2 de 3</p>  <!-- se anuncia -->
```

Así se viola (TurnoYa)

Camila toca "Reservar", la pantalla no cambia durante 6 segundos y vuelve a tocar: **se crean dos reservas**.

Así se cumple

El botón cambia de inmediato a "Reservando..." con un spinner, se deshabilita y al terminar aparece la confirmación con código.

H2 · Coincidencia entre el sistema y el mundo real

El sistema debe **hablar el lenguaje del usuario**, con palabras, conceptos y convenciones que le sean familiares, no términos internos del sistema. La información aparece en un orden natural y lógico.

En la práctica

- **Vocabulario del cliente**
  "Reserva tu cita" en lugar de "Booking de slot"; "Celular" en lugar de "MSISDN".
- **Formatos locales**
  S/ 35, "sáb 24 oct", "10:30 a. m.".

Así se viola (TurnoYa)

La lista de servicios muestra `SRV-001`, `SRV-002`… y el botón dice "Submit".

Así se cumple

"Corte clásico · 30 min · S/ 35" y un botón "Confirmar reserva".

Formatos locales con la API Intl

```
const soles = new Intl.NumberFormat('es-PE',
  { style: 'currency', currency: 'PEN' });
soles.format(35);                  // "S/ 35.00"
new Intl.DateTimeFormat('es-PE',
  { weekday: 'short', day: 'numeric', month: 'short' })
  .format(fecha);                  // "sáb, 24 oct"
```

H3 · Control y libertad del usuario

Los usuarios se equivocan a menudo y necesitan una **"salida de emergencia"** claramente marcada para abandonar la acción no deseada sin pasar por un proceso largo. Deshacer y rehacer.

En la práctica

- **Volver sin perder datos**
  El botón atrás conserva lo que se eligió en el paso anterior.
- **Deshacer**
  "Cita cancelada · Deshacer" durante unos segundos, mejor que un diálogo de confirmación.
- **Cerrar siempre**
  Todo modal se cierra con X, con Esc y tocando fuera.

Confirmar antes de perder datos

```
volver.addEventListener('click', () => {
  const hayDatos = nombre.value || hora;
  if (!hayDatos || confirm('¿Salir sin reservar?'))
    location.href = '/servicios';
});
// <dialog> nativo: Esc ya lo cierra (salida)
```

Así se viola (TurnoYa)

"CANCELAR" borra el formulario completo sin preguntar, y no hay forma de recuperarlo.

Así se cumple

"Volver a servicios" pide confirmación si hay datos; al cancelar una cita aparece "Deshacer".

H4 · Consistencia y estándares

Los usuarios no deberían preguntarse si **palabras, situaciones o acciones distintas significan lo mismo**. Consistencia interna (dentro de tu producto) y externa (con las convenciones de la plataforma y la industria).

En la práctica

- **Un componente, un aspecto**
  El botón primario se ve igual en todas las pantallas.
- **Una palabra, un significado**
  Si dices "cita", no cambies a "turno" o "reserva" en otra pantalla.
- **Convenciones de plataforma**
  Logo arriba a la izquierda lleva al inicio; el link subrayado es un link.

La consistencia vive en el design system

```
<!-- en todas las pantallas, la misma clase -->
<button class="btn">Confirmar reserva</button>
<button class="btn btn--link">Volver</button>
/* un solo lugar define cómo se ve */
.btn { background: var(--color-action-primary); }
```

Así se viola (TurnoYa)

Tres botones con tres estilos distintos; dos de ellos ("Submit" y "Reservar") hacen exactamente lo mismo.

Así se cumple

Un design system: `.btn` primario, `.btn--secondary` y `.btn--link`, iguales en todo el producto.

H5 · Prevención de errores

Mejor que un buen mensaje de error es un **diseño que evita que el error ocurra**. Elimina las condiciones propensas a errores o pide confirmación antes de una acción irreversible.

En la práctica

- **Restringe la entrada**
  Selector de fecha en lugar de texto libre; horarios ocupados deshabilitados.
- **Valida mientras escribe**
  Indica el formato esperado antes de enviar: "9 dígitos".
- **Valores por defecto sensatos**
  La fecha de hoy ya seleccionada; el profesional "Cualquiera".

Que el HTML impida el error

```
<input type="date" id="fecha" min="2026-10-25">
<input type="tel" inputmode="numeric"
       pattern="9[0-9]{8}" autocomplete="tel">
<input type="radio" name="hora" disabled>  <!-- ocupado -->
```

Así se viola (TurnoYa)

Fecha en un campo de texto: "dd/mm o mm/dd". El 03/04 puede ser 3 de abril o 4 de marzo.

Así se cumple

`<input type="date">` con la fecha de hoy y los días sin horarios deshabilitados.

H6 · Reconocer antes que recordar

Minimiza la **carga de memoria** del usuario haciendo visibles los elementos, acciones y opciones. El usuario no debería tener que recordar información de una parte de la interfaz a otra.

En la práctica

- **Resumen persistente**
  En el paso 3 se sigue viendo qué servicio, profesional y hora se eligió.
- **Labels siempre visibles**
  El placeholder desaparece al escribir: no sirve como label.
- **Historial y sugerencias**
  "Tu último servicio: Corte clásico".

Label visible, placeholder solo como ejemplo

```
<label for="nombre">Tu nombre</label>
<input id="nombre" placeholder="Ej. Camila Torres">
<aside class="resumen">
  Corte clásico · sáb 24 oct · 10:00 a. m.
</aside>
```

Así se viola (TurnoYa)

El placeholder "Nombre*" hace de label: al escribir, el campo ya no dice qué era.

Así se cumple

`<label>` visible encima del campo y un resumen fijo "Corte clásico · sáb 24 · 10:00 a. m.".

H7 · Flexibilidad y eficiencia de uso

Atajos y aceleradores, invisibles para el novato, que **aceleran al usuario experto**. El sistema sirve a ambos y permite personalizar acciones frecuentes.

En la práctica

- **Autocompletar**
  `autocomplete="name"` y `"tel"` llenan los datos en un toque.
- **Atajos**
  Repetir la última reserva.
- **Defaults inteligentes**
  Recordar el profesional preferido del cliente.

Autocompletado del navegador

```
<input autocomplete="name">
<input type="tel" autocomplete="tel">
<input type="email" autocomplete="email">
<!-- en el panel del negocio -->
document.addEventListener('keydown', e => {
  if (e.key === 'b') abrirBloquearHorario();
});
```

Así se viola (TurnoYa)

Camila, que reserva cada 2 semanas, debe escribir su nombre y celular cada vez.

Así se cumple

El navegador autocompleta sus datos y aparece "Repetir: Corte clásico con Raúl".

H8 · Diseño estético y minimalista

Las interfaces no deben contener **información irrelevante o rara vez necesaria**. Cada unidad extra de información compite con la relevante y reduce su visibilidad. Minimalista no significa vacío: significa enfocado.

En la práctica

- **Una acción primaria por vista**
  Lo demás es secundario y se ve secundario.
- **Divulgación progresiva**
  Mostrar lo esencial y dejar el detalle en "Ver más".
- **Sin ruido persuasivo**
  Ni banners que compiten ni cuentas regresivas falsas.

Divulgación progresiva nativa

```
<details>
  <summary>Ver políticas de cancelación</summary>
  <p>Puedes cancelar gratis hasta 2 horas antes.</p>
</details>
```

Así se viola (TurnoYa)

Banner "¡20 % OFF! se cierra en 15 s", tres botones y la letra pequeña con "cobros automáticos" en gris claro.

Así se cumple

Título, 5 campos, una acción primaria y las condiciones en texto legible.

H9 · Ayudar a reconocer, diagnosticar y recuperarse de errores

Los mensajes de error deben expresarse en **lenguaje sencillo** (sin códigos), **indicar con precisión el problema** y **sugerir una solución** de forma constructiva.

En la práctica

- **Qué pasó + cómo arreglarlo**
  "Ingresa un celular de 9 dígitos que empiece con 9".
- **Junto al campo**
  El error aparece debajo del campo, no en un alert ni arriba de la página.
- **Conserva lo escrito**
  Nunca borres el formulario por un error.

Error accesible y conectado al campo

```
<input id="cel" aria-invalid="true"
       aria-describedby="err-cel">
<p id="err-cel" aria-live="polite">
  Ingresa un celular de 9 dígitos que empiece con 9.
</p>
cel.focus();   // lleva al usuario al problema
```

Así se viola (TurnoYa)

"Error 0x45: VALIDATION_FAILED" arriba del formulario, y además se borró todo.

Así se cumple

Borde rojo + mensaje debajo del campo + foco en el primer error + datos intactos.

H10 · Ayuda y documentación

Lo ideal es que el sistema no necesite explicación. Si hace falta, la ayuda debe ser **fácil de encontrar, centrada en la tarea** y con pasos concretos, **en el momento** en que se necesita.

En la práctica

- **Ayuda contextual**
  Un hint junto al campo: "(te enviaremos la confirmación)".
- **Estados vacíos que enseñan**
  "No hay horarios este día. Prueba el lunes 26".
- **Ayuda siempre en el mismo lugar**
  Ayuda consistente.

Hint asociado al campo

```
<label for="cel">Celular</label>
<input id="cel" aria-describedby="hint-cel">
<p id="hint-cel" class="field__hint">
  Te enviaremos la confirmación por SMS.
</p>
```

Así se viola (TurnoYa)

Un link "Ayuda" que lleva a un PDF de 20 páginas.

Así se cumple

Hints cortos en cada campo, un estado vacío útil y "¿Dudas? Escríbenos por WhatsApp" siempre en el pie.

Cómo hacer una evaluación heurística, paso a paso

Es la técnica que aplicarás en el Taller 1. No necesita usuarios ni laboratorio: solo método. Cada evaluador revisa **por separado** y luego se consolidan los hallazgos.

1. **Define el alcance y la tarea**
   Qué pantallas y qué objetivo: "Camila reserva un corte para hoy".

2. **Primera pasada: recorre el flujo completo**
   Entiende la tarea de punta a punta sin anotar todavía.

3. **Segunda pasada: revisa cada heurística**
   Pantalla por pantalla, con la lista H1–H10 al lado.

4. **Registra cada hallazgo con evidencia**
   Qué viste y dónde, qué heurística o criterio viola, severidad 0–4 y propuesta.

5. **Consolida y prioriza**
   Une duplicados y ordena por severidad alta y esfuerzo bajo.

Un hallazgo bien escrito

✗ Así no

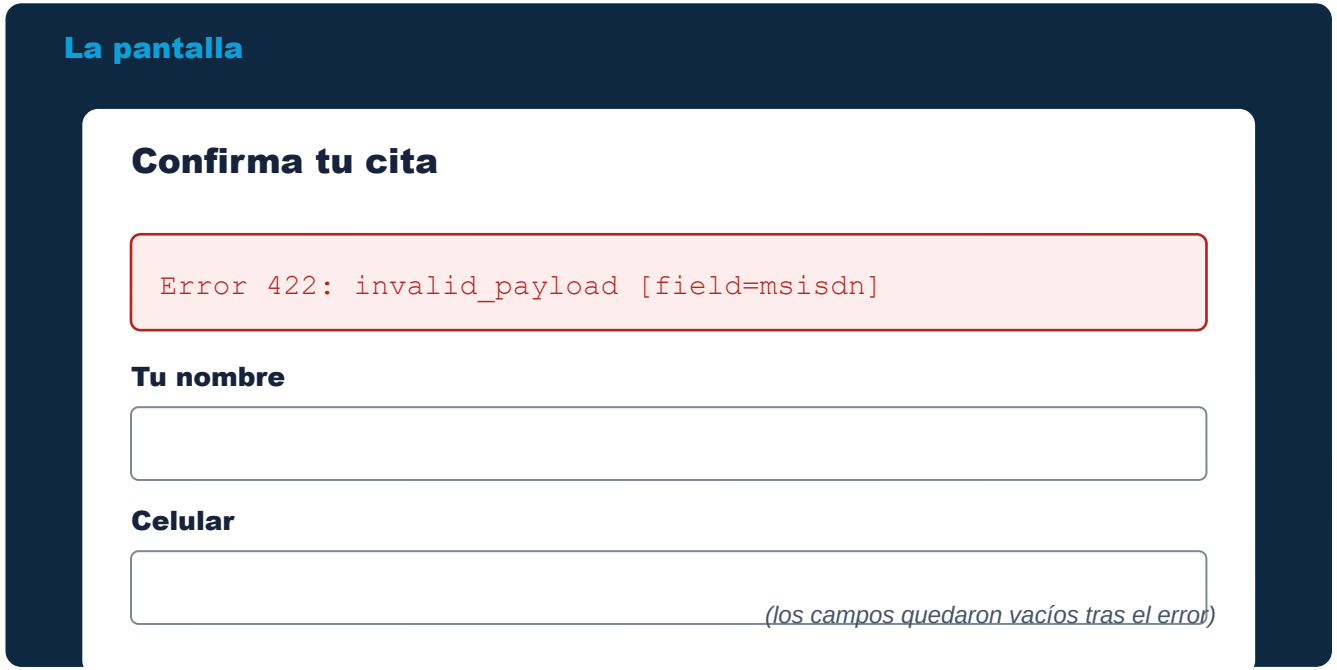
"Los colores están feos y el formulario es confuso."
No dice dónde, ni qué regla, ni cómo arreglarlo.

✓ Así sí

Evidencia: la etiqueta "Horarios" es #bbb sobre blanco (1.9:1).
Regla: WCAG 1.4.3 Contraste mínimo.
Severidad: 3 (mayor).
Propuesta: usar #52606d (6.5:1).

Reto relámpago · ¿Qué heurística se viola?

La pantalla

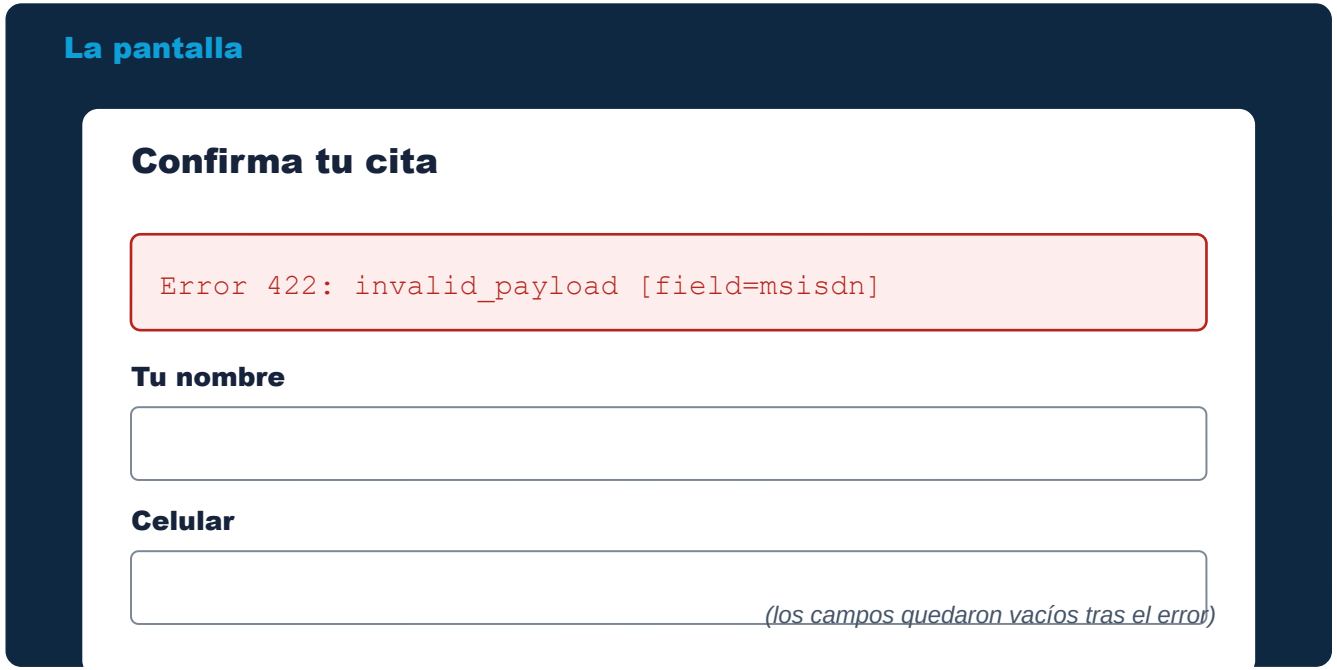


¿Cuál es la heurística MÁS afectada?

- **A** H1 · Visibilidad del estado del sistema
- **B** H4 · Consistencia y estándares
- **C** H9 · Reconocer, diagnosticar y recuperarse de errores
- **D** H10 · Ayuda y documentación

Reto relámpago · ¿Qué heurística se viola?

La pantalla



¿Por qué?

El mensaje usa un código técnico, no dice qué corregir y además **se borró lo escrito**: falla las tres partes de H9.

¿Cuál es la heurística MÁS afectada?

A. H1 · Visibilidad del estado del sistema

B. H4 · Consistencia y estándares

C. H9 · Reconocer, diagnosticar y recuperarse de errores

D. H10 · Ayuda y documentación

Ley de Fitts: tamaño y distancia

El tiempo para alcanzar un objetivo depende de la distancia hasta él y de su tamaño: más lejos y más pequeño = más lento y más errores.

$$T = a + b \cdot \log_2(1 + D / W)$$

Paul Fitts, 1954. T = tiempo, D = distancia al objetivo, W = ancho del objetivo, b = mide la velocidad del dispositivo (tiempo por bit), a = es el tiempo base de reacción del usuario, sin importar la distancia o el tamaño del objetivo

- **Duplicar el tamaño**
  reduce el tiempo tanto como reducir la distancia a la mitad.

Cómo se aplica en frontend

1. **CTA principal grande y cerca del pulgar**
   Botón a todo el ancho, abajo en móvil, 48 px de alto.
2. **Agranda el área clicable**
3. **Aleja lo destructivo**
   "Cancelar cita" lejos de "Confirmar" y más pequeño.

Área táctil cómoda

```
.btn { min-height: 48px; }       /* objetivo grande */
.cta-bar { position: fixed; bottom: 0; }  /* cerca */
<input type="radio" id="h1">
<label for="h1">10:00 a. m.</label>  <!-- área ×10 -->
```

Ley de Hick: cantidad de opciones

El tiempo para tomar una decisión aumenta con el número y la complejidad de las opciones.

$$T = b \cdot \log_2(n + 1)$$

William Hick y Ray Hyman, 1952. n = número de opciones igualmente probables. El crecimiento es logarítmico: cada opción extra pesa menos, pero suma.

- **No significa "menos es siempre mejor"**
  Significa organizar: agrupar, ordenar y destacar.
- **Aplica a decisiones, no a búsquedas**
  Si el usuario ya sabe lo que busca, una lista larga bien ordenada funciona.

Cómo se aplica en frontend

1. **Categorías antes que listas largas**
   Chips "Cortes · Barba · Combos" en lugar de un select de 24 servicios.
2. **Destaca la opción recomendada**
   "Más pedido" preseleccionado.
3. **Divulgación progresiva**
   Un paso a la vez: servicio → horario → datos.

Agrupar en lugar de listar

```
<div role="group" aria-label="Filtrar">
  <button aria-pressed="true">Todos</button>
  <button aria-pressed="false">Cortes</button>
  <button aria-pressed="false">Barba</button>
</div>
```

Ley de Miller: memoria de trabajo

La memoria de trabajo retiene pocos elementos a la vez. Agrupar la información en bloques (chunks) la hace manejable.

`7 ± 2  →  hoy se estima ~4 bloques`

George Miller, 1956 ("el número mágico siete, más o menos dos"). Estudios posteriores (Cowan, 2001) sitúan el límite real en alrededor de 4 bloques.

- **Mal uso frecuente**
  "Máximo 7 ítems en un menú": un menú no se memoriza, se lee. La ley trata de lo que hay que recordar.
- **Relación con H6**
  Si el usuario debe recordar algo de otra pantalla, ya perdiste.

Cómo se aplica en frontend

1. **Agrupa números largos**
   Celular "987 654 321" y código "TY-7K2QD", no "987654321".
2. **Formularios por secciones**
   Servicio / Horario / Tus datos, con encabezados.
3. **Resumen visible**
   No obligues a recordar lo elegido en el paso anterior.

Mostrar agrupado, guardar limpio

```
const mostrar = cel => cel.replace(
  /(\d{3})(\d{3})(\d{3})/, '$1 $2 $3');
mostrar('987654321');   // "987 654 321"
const limpio = valor.replace(/\s/g, '');
```

Ley de Jakob: las convenciones mandan

Los usuarios pasan la mayor parte de su tiempo en OTROS sitios. Por eso esperan que el tuyo funcione igual que los que ya conocen.

Jakob Nielsen, 2000. Está relacionada con la heurística H4 (consistencia externa).

- **Innova en tu propuesta de valor**
  no en dónde va el menú o cómo se elige una fecha.
- **Si cambias una convención**
  hazlo gradualmente y con un beneficio claro (y pruébalo con usuarios).

Cómo se aplica en frontend

1. **Usa controles nativos**
   `<select>`, `<input type="date">`: el usuario ya sabe usarlos y son accesibles.
2. **Ubicaciones esperadas**
   Logo arriba a la izquierda → inicio; usuario arriba a la derecha; acciones al final del formulario.
3. **Íconos universales**
   Lupa = buscar, X = cerrar, ← = volver. Con texto si hay duda.
4. **Patrones de la plataforma**
   En móvil, la navegación principal abajo (tab bar).

Lo nativo ya es conocido y accesible

```
<input type="date">          <!-- calendario del SO -->
<details><summary>Más</summary></details>
```

Umbral de Doherty y tiempos de respuesta

La productividad aumenta cuando el sistema responde en menos de 400 ms: el usuario mantiene el ritmo y no pierde la atención.

`< 0.1 s · < 1 s · < 10 s`

Walter Doherty y Ahrvind Thadani (IBM), 1982. Los 3 límites son de Jakob Nielsen (1993): 0.1 s se siente instantáneo, 1 s mantiene el flujo, 10 s es el límite de la atención.

- **Lo que importa es la percepción**
  Una espera con feedback se siente más corta que una sin él.
- **Más de 1 s**
  requiere indicador; más de 10 s, barra de progreso y opción de cancelar.

Cómo se aplica en frontend

1. **Feedback en < 100 ms**
   El botón cambia de estado apenas se toca.
2. **Skeleton screens**
   Mostrar la estructura mientras cargan los datos.
3. **UI optimista**
   Marcar el horario como elegido antes de que responda el servidor y revertir si falla.

UI optimista con reversión

```
slot.classList.add('is-selected');   // inmediato
try { await api.reservar(slot.id); }
catch { slot.classList.remove('is-selected');
        mostrarError('Ese horario acaba de ocuparse'); }
```

Reto relámpago · ¿Qué ley explica el problema?

Don Raúl nos escribe

"Mis clientes se demoran muchísimo en el primer paso. Tengo **24 servicios** en una sola lista desplegable, todos con el mismo formato, y muchos terminan llamándome para preguntar cuál les conviene."

¿Qué ley explica mejor la demora y qué harías?

- **A** Fitts · agrandar el select
- **B** Hick · agrupar en categorías y destacar el recomendado
- **C** Jakob · cambiar el select por un calendario
- **D** Doherty · cargar la lista más rápido

Reto relámpago · ¿Qué ley explica el problema?

Don Raúl nos escribe

"Mis clientes se demoran muchísimo en el primer paso. Tengo **24 servicios** en una sola lista desplegable, todos con el mismo formato, y muchos terminan llamándome para preguntar cuál les conviene."

¿Por qué?

El problema es **decidir** entre 24 opciones equivalentes (Hick). La solución: chips de categoría (Cortes · Barba · Combos), una etiqueta "Más pedido" y ver precio y duración en cada tarjeta. Así se aplica también H6 y el efecto Von Restorff.

¿Qué ley explica mejor la demora y qué harías?

A. Fitts · agrandar el select

B. Hick · agrupar en categorías y destacar el recomendado

C. Jakob · cambiar el select por un calendario

D. Doherty · cargar la lista más rápido

02

Accesibilidad web: WCAG 2.2

Qué exige el estándar, por qué importa y cómo se cumple desde el código

¿Qué es la accesibilidad y por qué es tu responsabilidad?

Accesibilidad web significa que **personas con discapacidad pueden percibir, entender, navegar e interactuar** con un sitio. En la práctica beneficia a todos: quien usa el celular bajo el sol, quien tiene el brazo enyesado o quien navega con teclado por rapidez.

1 de cada 9

personas en el mundo vive con una discapacidad significativa (OMS, 2023).

3 tipos

de limitación según el diseño inclusivo de Microsoft: permanente (un brazo), temporal (brazo roto) y situacional (un bebé en brazos).

Ley

En Perú, la Ley 29973 reconoce el derecho de las personas con discapacidad a acceder a la información. En la UE, la Ley Europea de Accesibilidad rige desde junio de 2025 y usa WCAG como referencia.

Discapacidades que el frontend debe considerar

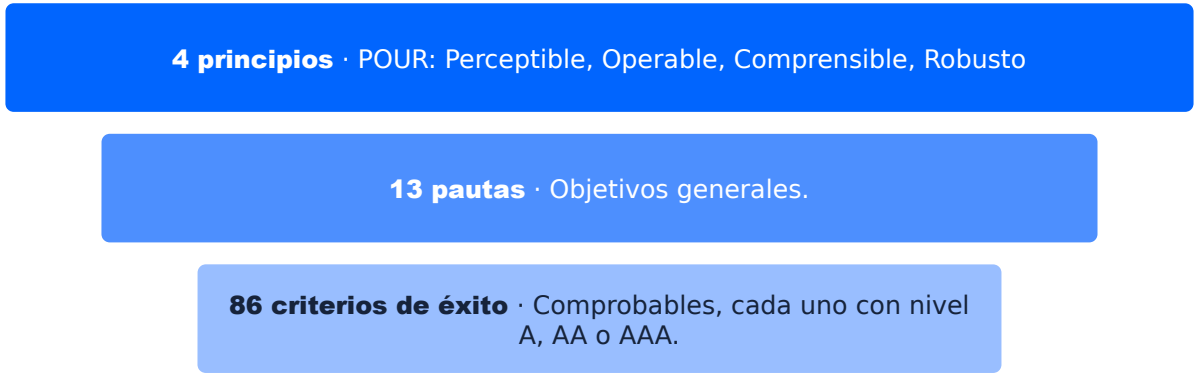
- **Visual** ceguera, baja visión, daltonismo
- **Auditiva** sordera, hipoacusia
- **Motriz** temblor, solo teclado, switch
- **Cognitiva** atención, memoria, dislexia

WCAG: el estándar internacional de accesibilidad web

Las **Web Content Accessibility Guidelines** las publica el **W3C** (a través de la Web Accessibility Initiative, WAI). Son neutrales a la tecnología: aplican a HTML, a apps híbridas y a documentos. Cada criterio es **comprobable**: se cumple o no se cumple.



Estructura del estándar



Cómo leer un criterio

1.4.3 = principio **1** (Perceptible) · pauta **4** (Distinguible) · criterio **3** (Contraste mínimo).

Cada criterio tiene:
- un **nivel** (A, AA o AAA)
- un documento **Understanding** que explica la intención
- **técnicas** suficientes y fallos comunes

WCAG 2.2 retiró el criterio 4.1.1 (Procesamiento) porque los navegadores ya resuelven esos errores de HTML.

Los 4 principios POUR

Si un sitio falla en uno de estos principios, hay personas que **no pueden usarlo**. Cada principio responde una pregunta concreta.

P Perceptible

¿Se puede percibir con al menos un sentido?

- Texto alternativo en imágenes
- Subtítulos en videos
- Contraste suficiente
- No depender solo del color

O Operable

¿Se puede usar con cualquier dispositivo de entrada?

- Todo funciona con teclado
- Foco visible y en orden lógico
- Tiempo suficiente
- Nada que parpadee más de 3 veces por segundo
- Objetivos táctiles de 24 px o más

C Comprensible

¿Se entiende el contenido y el funcionamiento?

- Idioma declarado
- Comportamiento predecible
- Labels e instrucciones
- Errores que explican cómo corregir
- No pedir el mismo dato dos veces

R Robusto

¿Lo interpretan bien navegadores y tecnologías de apoyo?

- HTML nativo válido
- Nombre, rol y valor en cada control
- Mensajes de estado

Niveles de conformidad: A, AA y AAA

Cada criterio tiene un nivel. Los niveles son **acumulativos**: cumplir AA significa cumplir todos los criterios A y todos los AA. La conformidad se declara por página completa, no por componente.

A — Mínimo

Sin esto, algunas personas no pueden usar el sitio en absoluto.

Ejemplos

- 1.1.1 Texto alternativo
- 2.1.1 Teclado
- 1.4.1 Uso del color
- 3.3.2 Etiquetas o instrucciones
- 4.1.2 Nombre, rol, valor

AA — Estándar recomendado

El objetivo de leyes, licitaciones y contratos en la mayoría de países. **Es nuestra meta en TurnoYa.**

Ejemplos

- 1.4.3 Contraste 4.5:1
- 1.4.4 Zoom al 200 %
- 1.4.10 Reflow a 320 px
- 2.4.7 Foco visible
- 2.5.8 Tamaño del objetivo 24 px

AAA — Mejorado

Máximo nivel. El W3C no recomienda exigirlo a sitios completos porque algunos contenidos no pueden cumplirlo.

Ejemplos

- 1.4.6 Contraste 7:1
- 2.4.13 Apariencia del foco
- 2.5.5 Objetivo de 44 px
- 3.1.5 Nivel de lectura

DIIC | INSTITUTE

Perceptible: que se pueda ver, oír o leer

La información y los componentes deben presentarse de forma que el usuario **pueda percibirlos**. Estos son los criterios que más afectan a una interfaz web.

1.1.1 Contenido no textual — A

Toda imagen informativa tiene `alt` con su significado. Las decorativas, `alt=""`. Los íconos-botón necesitan nombre.

1.3.1 Información y relaciones — A

La estructura visual existe en el código, listas y tablas reales.

1.4.1 Uso del color — A

El color no es el único medio para transmitir información.

1.4.3 Contraste (mínimo) — AA

Texto normal 4.5:1; texto grande (24 px, o 18.66 px en negrita) 3:1.

1.4.4 Cambio de tamaño del texto — AA

Con zoom al 200 % no se pierde contenido ni funcionalidad.

1.4.10 Reflow — AA

Manejo de scroll horizontal.

1.4.11 Contraste no textual — AA

Bordes de inputs, íconos y anillo de foco con al menos 3:1 frente a lo que los rodea.

1.4.12 Espaciado del texto — AA

Si el usuario aumenta interlineado y espaciado, nada se corta ni se superpone.

DMC | INSTITUTE

Contraste: cómo se calcula y qué valores cumplen

El contraste se mide como una **razón entre la luminancia relativa** de dos colores. Va de **1:1** (mismo color) a **21:1** (negro sobre blanco). No se estima a ojo: se calcula.

La fórmula (WCAG)

```
razón = (L1 + 0.05) / (L2 + 0.05)

L1 = luminancia del color más claro
L2 = luminancia del color más oscuro
```

Umbrales

- **4.5:1 · texto normal (AA)**
  El cuerpo, labels, ayudas y botones.
- **3:1 · texto grande y componentes (AA)**
  Texto ≥ 24 px o ≥ 18.66 px negrita; bordes, íconos, foco.
- **7:1 · texto normal (AAA)**
  Recomendado para contenido largo de lectura.

Ejemplos reales (texto normal)

Reservar	#999999 / #FFFFFF	**2.85:1**	✗ gris del Taller 1
Reservar	#767676 / #FFFFFF	**4.54:1**	✓ el gris más claro que pasa
Reservar	#52606D / #FFFFFF	**6.46:1**	✓ texto secundario TurnoYa
Reservar	#FFFFFF / #E53935	**4.23:1**	✗ botón rojo del Taller 1
Reservar	#FFFFFF / #0B6E75	**6.00:1**	✓ botón primario TurnoYa
Reservar	#1F2933 / #FFFFFF	**14.76:1**	✓ texto principal TurnoYa

DIIC | INSTITUTE

Operable: que se pueda usar con cualquier dispositivo

Los componentes y la navegación deben poder **operarse con teclado, mouse, toque o tecnologías de apoyo**. WCAG 2.2 agregó aquí 3 criterios AA nuevos (marcados con ★).

2.1.1 Teclado — A

Toda funcionalidad se puede usar con teclado.

2.1.2 Sin trampas de teclado — A

Si el foco entra en un componente (modal, mapa), también puede salir.

2.2.1 Tiempo ajustable — A

Si hay límite de tiempo, se puede apagar, ajustar o extender.

2.4.3 Orden del foco — A

El foco recorre la página en un orden lógico, el mismo de lectura.

2.4.7 Foco visible — AA

Siempre se ve qué elemento tiene el foco.

2.4.11 ★ Foco no oculto (mínimo) — AA

El elemento enfocado no queda tapado por completo por un header fijo, un banner o un CTA fijo.

2.5.7 ★ Movimientos de arrastre — AA

Todo lo que se hace arrastrando tiene alternativa con un solo toque.

2.5.8 ★ Tamaño del objetivo (mínimo) — AA

Objetivos táctiles de al menos 24 × 24 px CSS, o con espacio suficiente alrededor.

DIIC | INSTITUTE

Comprensible: que se entienda qué es y qué pasa

La información y el funcionamiento de la interfaz deben ser **entendibles y predecibles**. Aquí viven los criterios de formularios: los que más impactan en una app de reservas.

3.1.1 Idioma de la página — A

`<html lang="es">`: el lector de pantalla elige la voz y la pronunciación correctas.

3.2.2 Al introducir datos — A

Cambiar un campo no provoca un cambio de contexto inesperado (enviar el form, cambiar de página).

3.3.1 Identificación de errores — A

El error se detecta y se describe en texto, identificando el campo.

3.3.2 Etiquetas o instrucciones — A

Cada campo tiene un label visible e indica el formato esperado.

3.3.3 Sugerencia ante errores — AA

Si se sabe cómo corregir, se dice: "Ingresa 9 dígitos que empiecen con 9".

3.2.6 ★ Ayuda consistente — A

Si hay ayuda (chat, teléfono), aparece en el mismo lugar relativo en todas las páginas.

3.3.7 ★ Entrada redundante — A

No pedir dos veces el mismo dato en un proceso: se precarga o se permite elegirlo.

3.3.8 ★ Autenticación accesible — AA

El login no exige resolver acertijos ni memorizar; se permite pegar y usar gestores de contraseñas.

Qué trajo WCAG 2.2: 9 criterios nuevos

Publicada como recomendación del W3C en **octubre de 2023**. Pone el foco en usuarios con **baja visión, discapacidad motriz y cognitiva**, sobre todo en dispositivos móviles.

2.4.11 · AA

Foco no oculto (mínimo)
El foco no queda totalmente tapado por contenido fijo.

2.4.12 · AAA

Foco no oculto (mejorado)
No queda tapado ni parcialmente.

2.4.13 · AAA

Apariencia del foco
Indicador de foco grande y con buen contraste.

2.5.7 · AA

Movimientos de arrastre
Alternativa de un solo toque a todo arrastre.

2.5.8 · AA

Tamaño del objetivo (mínimo)
24 × 24 px CSS o espacio equivalente.

3.2.6 · A

Ayuda consistente
La ayuda, siempre en el mismo lugar.

3.3.7 · A

Entrada redundante
No pedir dos veces el mismo dato.

3.3.8 · AA

Autenticación accesible (mínimo)
Sin pruebas cognitivas para iniciar sesión.

3.3.9 · AAA

Autenticación accesible (mejorado)
Ni siquiera reconocer objetos o imágenes.

Retirado: 4.1.1 Procesamiento (parsing). Los navegadores modernos ya corrigen esos errores de HTML.

Cómo probar la accesibilidad: manual + automático

Las herramientas automáticas detectan **solo una parte** de los problemas (contraste, labels faltantes, atributos inválidos). Que el foco tenga sentido o que un mensaje se entienda lo verifica una persona. Se combinan siempre.

Pruebas manuales (5 minutos por pantalla)

1. **Teclado**
   Recorre todo con Tab / Shift+Tab. ¿Ves el foco? ¿Llegas a todo? ¿El orden es lógico?

2. **Zoom 200 %**
   `Ctrl` + `+` cuatro veces. ¿Se lee y funciona todo?

3. **Reflow 320 px**
   DevTools → vista móvil a 320 px. ¿Hay scroll horizontal?

4. **Lector de pantalla**
   Windows: **Narrador** (`Ctrl + Win + Enter`) o **NVDA** (gratis). ¿Se anuncian labels, estados y errores?

5. **Sin color**
   DevTools → Rendering → Emulate vision deficiencies → Achromatopsia.

Herramientas automáticas

Lighthouse
Integrado en Chrome y Edge (F12 → Lighthouse). Da un puntaje de 0 a 100 y la lista de fallos.

axe DevTools
Extensión gratuita de Deque. Pocos falsos positivos; indica el criterio WCAG de cada fallo.

WAVE
Extensión de WebAIM. Dibuja sobre la página dónde está cada problema.

03

Taller 1 · Auditoría UX/UI y corrección accesible

Diagnosticar con heurísticas y WCAG, y corregir en código lo que impide reservar

Parte A · Las 4 pasadas de auditoría

Cada pasada mira la pantalla con un objetivo distinto. Así no se escapan problemas que solo aparecen con teclado, en móvil o midiendo contraste.

1 · Visual
3 min

- ¿Entiendo qué hace y qué botón presionar? (H2, H8)
- Si elijo un horario, ¿se ve elegido? (H1)
- ¿Hay botones iguales que hacen cosas distintas? (H4)
- Envía el formulario vacío: ¿qué pasa con lo escrito? (H5, H9)

2 · Teclado
2 min

- Clic en la barra de direcciones y luego solo Tab
- ¿Ves siempre dónde está el foco? (2.4.7)
- ¿Llegas a los horarios y eliges uno? (2.1.1)
- ¿El orden del foco es el de lectura? (2.4.3)

3 · Móvil y zoom
2 min

- F12 → Ctrl + Shift + M → 360 px
- ¿Se lee? ¿Hay scroll horizontal? (1.4.10)
- ¿Cada cosa se toca sin errar? (2.5.8)
- Ctrl + '+' hasta 200 %: ¿se pierde algo? (1.4.4)

4 · Contraste y herramientas
3 min

- Selector de color de DevTools: ratio de "Horarios" (1.4.3)
- ¿Algún estado depende solo del color? (1.4.1)
- Revisa qué criterios marca como fallidos

Cómo usar DevTools para auditar (Chrome o Edge)

Tres herramientas que ya tienes instaladas y que resuelven el 80 % de una auditoría rápida.

Medir contraste

1. F12 → pestaña **Elements**
2. Selecciona el texto con el ícono de flecha (`Ctrl + Shift + C`)
3. En **Styles**, clic en el cuadrado de color de `color`
4. Lee **Contrast ratio**: ✓ AA / ✓ AAA o ✗

Vista móvil

1. F12 → `Ctrl + Shift + M`
2. Arriba elige **Dimensions: Responsive**
3. Escribe el ancho: 360, 768, 1280
4. En la **Console**: `document.documentElement.scrollWidth`

DWC | INSTITUTE

Manos a la obra · Parte A: diagnosticar

5 MINUTOS

1. **Abre `taller-1-auditoria/inicio/reserva.html` con Live Server**
   No modifiques el código todavía: primero se diagnostica.

2. **Haz las 4 pasadas**
   Visual (3 min) · Teclado (2 min) · Móvil y zoom (2 min) · Contraste (3 min).

3. **Registra cada hallazgo en `auditoria.md`**
   Pasada · evidencia (qué y dónde) · heurística o criterio WCAG · severidad 0–4 · propuesta.

4. **Elige tu top 3**
   Severidad alta y esfuerzo bajo primero. Pégalo en el chat de la clase.

Entregable

- Mínimo **8 hallazgos** con evidencia
- Tu **top 3** priorizado

Pista

Hay **18 problemas**. Prueba enviar el formulario sin nombre y mira qué pasa con el celular que habías escrito.

Solución Parte A · 18 hallazgos agrupados por corrección

T1-1 · Estructura y lectura

- Zoom bloqueado y ancho fijo de 1024 · 1.4.4 · sev 4
- Textos #999/#bbb/#ccc: 2.8, 1.9 y 1.6:1 · 1.4.3 · sev 3
- Placeholder como label · 3.3.2 · H6 · sev 3
- Sin `lang` en `<html>` · 3.1.1 · sev 2
- "Booking de Slot", "Submit", `SRV-001` · H2 · sev 3

T1-2 · Operable

- `outline:none`: foco invisible · 2.4.7 · sev 4
- Horarios en `<span>`: sin teclado · 2.1.1 · 4.1.2 · sev 4
- Elegir horario no da feedback · H1 · sev 4
- Libre vs ocupado solo por color · 1.4.1 · sev 3
- Objetivos de 16–20 px · 2.5.8 · Fitts · sev 3

T1-3 · Errores

- "Error 0x45: VALIDATION_FAILED" · H9 · 3.3.1 · sev 3
- El error borra todo lo escrito · H5 · H3 · sev 4
- Error no asociado ni anunciado · 4.1.2 · sev 3
- Fecha en texto "dd/mm o mm/dd" · H5 · sev 3

T1-4 · Estado del sistema

- 6 s sin feedback y `alert('OK')` · H1 · Doherty · sev 3
- 3 botones, 2 iguales, destructivo destacado · H4 · H8 · sev 3
- CANCELAR borra sin preguntar · H3 · sev 3
- Cuenta regresiva y letra pequeña · 2.2.1 · H8 · sev 3

Manos a la obra · Parte B: corregir en código

Trabaja sobre `inicio/reserva.html`. Las 4 diapositivas siguientes explican cada corrección parte por parte. Si te atrasas, copia el bloque marcado `[T1-n]` de `final/reserva.html` y **lee por qué funciona**.

10
MINUTOS

T1-1 — 3 min

lang, labels, contraste, lenguaje

T1-2 — 4 min

foco visible, radio buttons

T1-3 — 3 min

errores accesibles sin borrar datos

T1-4 — 2 min

estado de carga y confirmación

Cómo sabes que terminaste

- Reservas usando **solo el teclado** (Tab, flechas, Espacio, Enter)
- Al enviar vacío ves 3 mensajes claros y **no se borra nada**

reserva.html — idioma, viewport, contraste y labels (T1-1)

taller-1-auditoria/final/reserva.html · [T1-1]

```
<!-- ANTES: <html>  y  width=1024, user-scalable=no -->
<html lang="es">
<meta name="viewport"
      content="width=device-width, initial-scale=1">

/* ANTES: color:#999 (2.85:1) y #bbb (1.92:1) */
body { color: #1f2933; }   /* 14.8:1 */
.sub { color: #52606d; }   /* 6.5:1  */

<!-- ANTES: <input placeholder="Nombre*"> -->
<label for="servicio">Servicio</label>
<select id="servicio">
  <option>Corte clásico · 30 min · S/ 35</option>
</select>

<label for="nombre">Tu nombre</label>
<input id="nombre" autocomplete="name"
       placeholder="Ej. Camila Torres">
```

¿Qué hace cada parte?

1. **lang="es"**
   El lector de pantalla usa voz y reglas de pronunciación en español (WCAG 3.1.1).

2. **Viewport correcto**
   `device-width` adapta el ancho al celular; sin `user-scalable=no` el usuario puede hacer zoom (1.4.4).

3. **Colores con contraste**
   Texto principal 14.8:1 y secundario 6.5:1. Ambos superan 4.5:1 (1.4.3).

4. **label + for**
   El `for` apunta al `id`: el lector anuncia "Servicio" y el clic en el label enfoca el campo (1.3.1, 3.3.2).

5. **Lenguaje del cliente**
   Nombre, duración y precio en lugar de `SRV-001` (H2).

6. **Placeholder como ejemplo**
   El label no desaparece al escribir; `autocomplete` acelera (H6, H7).

reserva.html — foco visible y horarios accesibles (T1-2)

taller-1-auditoria/final/reserva.html · [T1-2]

```
/* ANTES: input, select { outline: none; } */
:focus-visible { outline: 3px solid #1d4ed8;
                 outline-offset: 2px; }

<fieldset>
  <legend>Horario</legend>
  <div class="slot">
    <input type="radio" name="hora" id="h0" value="9:00 a. m.">
    <label for="h0">9:00 a. m.</label>
  </div>
  <div class="slot">
    <input type="radio" name="hora" id="h1" disabled>
    <label for="h1">9:30 a. m.<span class="visually-hidden">
      (ocupado)</span></label>
  </div>
</fieldset>
.slot label { min-height: 44px; }
.slot input:checked + label::before { content: "✓"; }
```

¿Qué hace cada parte?

1. **:focus-visible**
   El anillo aparece al navegar con teclado y no al hacer clic con el mouse. Azul 6.7:1 (2.4.7, 1.4.11).

2. **fieldset + legend**
   Agrupa los radios: el lector anuncia "Horario, grupo" antes de cada opción (1.3.1).

3. **Radio nativo**
   Tab entra al grupo y las flechas cambian de opción. Rol, nombre y estado sin ARIA (2.1.1, 4.1.2).

4. **El label es el botón visible**
   El input se oculta visualmente; el label es la zona táctil y se pinta según `:checked`.

5. **disabled + texto oculto**
   El ocupado sale del orden de foco y el lector oye "(ocupado)": no depende del color (1.4.1).

6. **44 px y ✓**
   Área táctil cómoda (2.5.8, Fitts) y un símbolo que confirma la selección (H1).

reserva.html — errores que ayudan y no borran (T1-3)

taller-1-auditoria/final/reserva.html · [T1-3]

```
1  <input id="cel" type="tel" aria-describedby="err-cel">
   <p class="error" id="err-cel" aria-live="polite"></p>

   // Escribe el mensaje y marca el campo ('' = sin error)
2  function marcar(campo, idError, mensaje) {
     document.getElementById(idError).textContent = mensaje;
3    campo.setAttribute('aria-invalid', mensaje ? 'true' : 'false');
     return !mensaje;
   }
   form.addEventListener('submit', (e) => {
     e.preventDefault();                    // no recargar la página
4    const okCel = marcar(cel, 'err-cel',
       /^9\d{8}$/.test(cel.value.replace(/\s/g, ''))
5        ? '' : 'Ingresa un celular de 9 dígitos que empiece con 9.');
     // ANTES: limpiar();  ← borraba todo lo escrito
     if (!okNombre) { nombre.focus(); return; }
6    if (!okCel)    { cel.focus();    return; }
   });
```

¿Qué hace cada parte?

1. **Error asociado al campo**
   `aria-describedby` hace que el lector lea el mensaje al enfocar el campo; `aria-live` lo anuncia al aparecer (4.1.3).

2. **Una función, un patrón**
   `marcar()` escribe el texto y pone `aria-invalid`, que además pinta el borde rojo con CSS.

3. **aria-invalid**
   Comunica el estado "inválido" a las tecnologías de apoyo (4.1.2).

4. **Validación tolerante**
   Se quitan espacios antes de validar: "987 654 321" es válido (Tesler).

5. **Mensaje que dice cómo corregir**
   Qué pasó y qué hacer (H9, WCAG 3.3.1 y 3.3.3).

6. **Foco al primer error, datos intactos**
   Nada se borra (H5, H3) y el usuario llega directo al problema.

reserva.html — estado de carga y confirmación (T1-4)

taller-1-auditoria/final/reserva.html · [T1-4]

```
// ANTES: setTimeout(() => alert('OK'), 6000);  sin feedback
btn.disabled = true;                          // evita el doble envío
btn.setAttribute('aria-busy', 'true');
btn.innerHTML =
  '<span class="spinner" aria-hidden="true"></span>Reservando…';

setTimeout(() => {                            // simula la respuesta
  document.getElementById('form-card').hidden = true;
  const ok = document.getElementById('success');
  ok.hidden = false;
  ok.focus();                                 // se anuncia el éxito
}, 1200);

<section id="success" hidden tabindex="-1">
  <h1>¡Cita reservada!</h1>  <p class="code">TY-7K2QD</p>
</section>

@media (prefers-reduced-motion: reduce) {
  .spinner { animation: none; } }
```

¿Qué hace cada parte?

1. **Bloquear el doble envío**
   `disabled` impide que Camila toque dos veces y cree dos reservas (H1, H5).

2. **Feedback inmediato**
   El texto cambia en el mismo instante del clic: dentro del umbral de Doherty. El spinner es decorativo (`aria-hidden`).

3. **Respuesta del servidor**
   En la Sesión 4 esto será una llamada real a un servicio con su contrato (DTO).

4. **Mover el foco**
   Al mostrar el éxito, el foco va ahí y el lector anuncia "¡Cita reservada!".

5. **tabindex="-1"**
   Permite enfocar la sección por código sin agregarla al recorrido de Tab.

6. **Movimiento reducido**
   Respeta a quien configuró su sistema para evitar animaciones.

Resultado esperado (Taller 1)

Al enviar vacío

TurnoYa Servicios Mis citas Ayuda

Reserva tu cita

Barbería Don Raúl · Miraflores

Servicio

Corte clásico · 30 min · S/ 35

Fecha

10/26/2026

Horario (los tachados ya están ocupados)

9:00 a. m.	~~9:30 a. m.~~
10:00 a. m.	10:30 a. m.

Al confirmar

TurnoYa Servicios Mis citas Ayuda

Reserva tu cita

Barbería Don Raúl · Miraflores

Servicio

Corte clásico · 30 min · S/ 35

Fecha

10/26/2026

Horario (los tachados ya están ocupados)

9:00 a. m.	~~9:30 a. m.~~
10:00 a. m.	10:30 a. m.

Cómo verificar que funcionó

- Reservas solo con teclado
- Al enviar vacío: 3 mensajes y nada borrado
- Lighthouse Accessibility ≥ 95
- axe DevTools: 0 problemas WCAG AA
- A 360 px: sin scroll horizontal

Qué aprendiste en este taller

- Auditar con un protocolo de 4 pasadas
- Escribir hallazgos con evidencia y severidad
- Que lo nativo (`label`, `radio`, `fieldset`) resuelve la mayoría
- ARIA mínimo: `aria-invalid`, `describedby`, `live`, `busy`

Receso · 10 minutos

Pausa activa. Al volver: Components & Design Systems y Taller 2

04

Components & Design Systems

Las bases de toda interfaz y el contexto más valioso que le puedes dar a una IA

¿Qué es un componente de UI?

Un componente es una **pieza de interfaz reutilizable** que encapsula estructura (HTML), apariencia (CSS) y comportamiento (JS) detrás de una **API pequeña**. Se diseña una vez y se usa en todas las pantallas.

La API de un componente

- **Props / atributos**
  Los datos que recibe: texto, `type`, `href`.
- **Variantes**
  Versiones visuales con el mismo propósito: primary, secondary, danger, link.
- **Estados**
  Cómo cambia por la interacción o los datos: hover, focus, active, disabled, loading, error.
- **Contenido (slots)**
  Lo que se le inserta: un ícono antes del texto, el cuerpo de una tarjeta.
- **Eventos**
  Lo que comunica hacia afuera: `click`, `change`, `submit`.

```
<!-- HTML + CSS: clase base · variante · estado -->
<button class="btn btn--secondary" disabled>
  Ver detalle
</button>

// la misma API como props
<Button variant="secondary" disabled>
  Ver detalle
</Button>
```

Un buen componente es…

Accesible por defecto (usa el elemento nativo), **agnóstico al contenido** (no sabe si dice "Reservar" o "Pagar") y **configurable solo por su API** (nadie le sobrescribe estilos desde afuera).

¿Qué es un design system (y qué no lo es)?

Un design system es el **conjunto de decisiones, piezas y reglas** que permite construir productos consistentes a escala: incluye los componentes, pero también los tokens que los alimentan, los patrones que los combinan y la documentación que explica cuándo usar cada uno.

Principios	Qué valoramos: "claridad antes que estética", "accesible por defecto".
Design tokens	Las decisiones visuales como datos: color, tipografía, espaciado.
Componentes	Button, Field, Card… con variantes, estados y código.
Patrones	Combinaciones probadas: formulario de reserva, estado vacío, flujo de 3 pasos.
Documentación y gobierno	Cuándo usar cada pieza, quién aprueba cambios, cómo se versiona.

No es lo mismo…

- **Guía de estilo**
  Colores, logo y tipografías en un PDF. Sin código ni componentes.
- **UI kit**
  Componentes dibujados en Figma, sin implementación.
- **Librería de componentes**
  Código reutilizable, pero sin principios ni reglas de uso.

Atomic Design: de lo más pequeño a la página

Metodología propuesta por **Brad Frost** (2013) que organiza la interfaz en 5 niveles inspirados en la química. Ayuda a decidir **dónde vive cada estilo** y en qué orden construir.

Átomos	Moléculas	Organismos	Templates	Páginas
La pieza mínima con sentido propio.	Átomos que funcionan juntos.	Secciones completas de la interfaz.	La estructura de una pantalla, sin datos reales.	El template con contenido real.
Button, Input, Label, Badge	**Field (label + input + error), Slot**	**SlotGroup, ServiceCard, Header**	**BookingLayout: pasos + contenido + CTA**	**"Elige tu horario" con los horarios de hoy**

Regla de dependencia

Una página **no define estilos**: solo compone organismos. Si cada nivel solo usa el nivel de abajo, el resultado es consistente aunque lo genere una IA.

Design tokens: las decisiones de diseño como datos

Un token es una **decisión de diseño con nombre**, guardada como dato e independiente de la plataforma. En lugar de escribir `#0b6e75` en 29 lugares, escribes `color.action-primary` y la decisión vive en **un solo sitio**.

Tipos de tokens

Color
texto, fondos, bordes, estados

Tipografía
familia, tamaño, peso, interlineado

Espaciado
márgenes, paddings, gaps

Radio
esquinas de botones y tarjetas

Sombra / elevación
profundidad de capas

Movimiento
duración y curvas de animación

Tamaño
alto de controles, íconos, áreas táctiles

Breakpoints y z-index
puntos de quiebre y capas

tokens.json · formato W3C (DTCG)

```
{
  "color": {
    "teal-600": { "$type": "color", "$value": "#0b6e75" },
    "action-primary": {
      "$type": "color",
      "$value": "{color.teal-600}"
    }
  }
}
```

El **Design Tokens Community Group** del W3C definió este formato JSON. Herramientas como **Style Dictionary** lo transforman en variables CSS, Swift, Android o variables de Figma.

Las 3 capas de tokens y quién cambia cada una

Separar los tokens en capas permite cambiar **una decisión sin tocar las demás**. La regla es una sola: **cada capa solo puede referenciar a la capa de abajo**.

CAPA 1 · PRIMITIVOS

¿Qué valores existen?

```
--teal-600: #0b6e75;
--gray-900: #1f2933;
--space-4: 16px;
```

Cambia en un **rebranding**: nueva paleta, nueva escala.

↑

CAPA 2 · SEMÁNTICOS

¿Para qué se usa?

```
--color-action-primary: var(--teal-600);
--color-text: var(--gray-900);
```

Cambia en un **tema**: modo oscuro, alto contraste, marca blanca.

↑

CAPA 3 · COMPONENTE

¿Dónde se aplica?

```
--button-bg: var(--color-action-primary);
--button-height: 48px;
```

Cambia al **ajustar un componente** sin afectar a los demás.

Los componentes consumen **semánticos** o **de componente**, nunca primitivos. Si en `components.css` aparece `--teal-600`, alguien se saltó una capa.

Cómo nombrar tokens: por intención, no por apariencia

El nombre de un token es un **contrato**: lo leen diseñadores, desarrolladores y, desde la Sesión 2, la IA. Un buen nombre dice **para qué sirve**, no cómo se ve hoy.

Patrón para semánticos

color	**action**	**primary**	**hover**
categoría	rol	variante	estado

`--color-action-primary-hover`

- **kebab-case y en inglés o español, pero siempre igual**
  Mezclar `--colorPrimary` y `--color-texto` confunde a personas y a la IA.
- **Escalas numéricas en primitivos**
  `--gray-50` (claro) a `--gray-950` (oscuro); `--space-1` = 4 px.
- **No codifiques el valor en el nombre**
  `--space-16px` deja de ser cierto el día que cambie.

✗ Evita

```
--blue
--color-1
--rojo-bonito
--btn-big-green
--text-999
--space-13
```

✓ Prefiere

```
--color-action-primary
--color-text-muted
--color-danger
--button-height
--color-border
--space-3
```

Anatomía de un componente: el botón

Un botón parece trivial, pero tiene **partes, variantes y al menos 6 estados**. La IA suele generar solo default y hover: pedir el resto es parte de tu trabajo.



Default	Hover	Focus	Active	Disabled	Loading
`:default`	`:hover`	`:focus-visible`	`:active`	`:disabled`	`.is-loading`

Partes	contenedor · label · ícono opcional · spinner
Variantes	primary · secondary · danger · link (cambian la apariencia, no el propósito)
Tamaños	sm · md · lg, o `--block` para ancho completo
Reglas	un solo primary por vista · texto con verbo ("Confirmar reserva") · 44–48 px de alto en móvil

Variante ≠ estado. Error clásico: crear una "variante disabled". Disabled es un estado que debe funcionar en **todas** las variantes.

Estados de UI: una pantalla no tiene un solo diseño

UI stack: toda pantalla que muestra datos pasa por varios estados. Si el diseño (o el prompt) solo describe el ideal, el usuario verá pantallas en blanco o rotas.

Ideal

Hay datos y todo funciona.
TurnoYa: La grilla con 9 horarios.

Vacío

No hay datos todavía. Es una oportunidad para guiar.
TurnoYa: "No hay horarios este día. Prueba el lunes 26".

Cargando

Los datos vienen en camino.
TurnoYa: Skeleton de 9 rectángulos grises.

Error

Algo falló. Explica y ofrece una salida.
TurnoYa: "No pudimos cargar los horarios. Reintentar".

Parcial

Hay algunos datos, no todos.
TurnoYa: Solo 1 horario libre: "¡Último horario de hoy!".

Éxito

Confirmación de que la acción terminó.
TurnoYa: "¡Cita reservada!" con código TY-7K2QD.

Por qué el design system es tu mejor prompt

Un modelo de IA genera **lo más probable** según lo que ha visto. Sin contexto, eso es un diseño genérico y distinto en cada intento. Con un design system, la IA **solo ensambla piezas que ya aprobaste**.

Sin design system

- Cada pantalla inventa un color nuevo
- Botones con 3 estilos
- Estados olvidados
- Revisión lenta: no hay contra qué comparar

Con design system

- La IA usa `var(--color-action-primary)`
- Reutiliza `.btn` y sus variantes
- Los estados ya están definidos
- Revisas contra reglas explícitas

Cómo se conecta (Sesiones 2 y 3)

- `CLAUDE.md`: "usa solo tokens semánticos"
- `tokens.css` y `components.css` en el repo
- Variables de Figma con los mismos nombres
- MCP server de Figma: el agente lee el diseño

Lo que construyes hoy en el Taller 2 es, literalmente, el contexto que Claude Code leerá en la Sesión 2.

Reto relámpago · ¿Qué está mal en este componente?

components.css

```css
.card {
  background: var(--gray-0);
  border-radius: var(--card-radius);
  padding: var(--space-6);
  box-shadow: var(--shadow-1);
}
```

¿Qué problema tiene?

A Nada, todos los valores son tokens

B Usa un primitivo (`--gray-0`) dentro de un componente

C Debería escribir `#ffffff` para ser más claro

D `--card-radius` debería ser `--radius-md`

Reto relámpago · ¿Qué está mal en este componente?

components.css

```css
.card {
  background: var(--gray-0);
  border-radius: var(--card-radius);
  padding: var(--space-6);
  box-shadow: var(--shadow-1);
}
```

¿Qué problema tiene?

- **A** Nada, todos los valores son tokens
- **B** Usa un primitivo (`--gray-0`) dentro de un componente
- **C** Debería escribir `#ffffff` para ser más claro
- **D** `--card-radius` debería ser `--radius-md`

¿Por qué?

Se saltó la capa semántica. En modo oscuro `--gray-0` seguiría siendo blanco y la tarjeta quedaría brillando. Debe ser `var(--card-bg)` o `var(--color-surface)`.

05

Taller 2 · Design tokens y componentes

De 11 colores repetidos 29 veces a un design system mínimo con tema oscuro

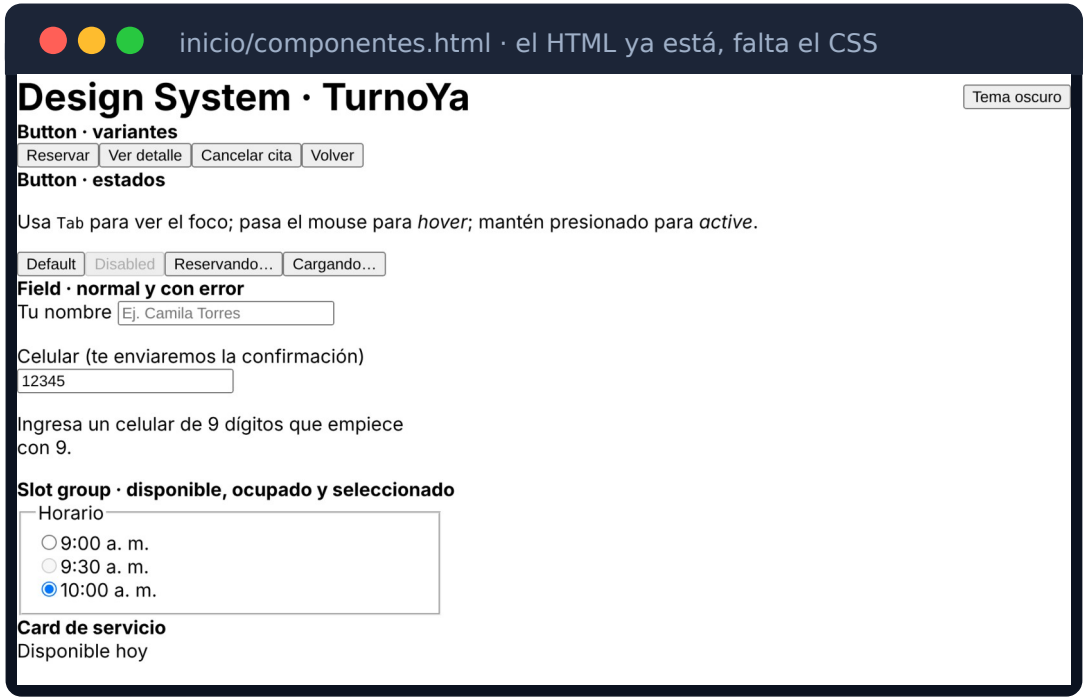
Diseño previo · ¿qué vamos a construir y por qué?

OBJETIVO

Que TurnoYa tenga **una sola fuente de verdad visual**: cambiar la marca o el tema en un archivo, sin buscar y reemplazar colores en todo el proyecto.

Lo que construimos (en orden)

1. **T2-1** Inventario de los colores de `reserva.html` y su intención.
2. **T2-2** Primitivos: paleta, espaciado, radios y tipografía.
3. **T2-3** Semánticos: roles de color + tema oscuro (bonus).
4. **T2-4** Tokens de componente: botón, campo, tarjeta.
5. **T2-5 / T2-6** Button: base, variantes y 6 estados.
6. **T2-7** Field, slot, card y badge con tokens.
7. **T2-8** Refactor de `reserva.html` (si da el tiempo).



Archivos del taller: + nuevo ~ modificado

`~ styles/tokens.css` `~ styles/components.css` `+ componentes.html` `~ reserva.html`

T2-1 · Inventario: cada color tiene una intención

En `inicio/reserva.html` presiona `Ctrl + F` y busca `#`. Hay **11 colores distintos repetidos 29 veces**. Para cada uno anota **dónde aparece** y **para qué se usa**: esa intención es el nombre del token semántico.

`#0b6e75`	botón, slot elegido, links	`--color-action-primary`
`#095a60`	hover del botón	`--color-action-primary-hover`
`#1f2933`	texto principal	`--color-text`
`#52606d`	subtítulo, hints, condiciones	`--color-text-muted`
`#7b8794`	borde de campos y slots	`--color-border`
`#e4e7eb`	línea del header	`--color-divider`

`#f5f7fa`	fondo de la página	`--color-bg`
`#ffffff`	tarjeta, campos, texto en botón	`--color-surface / --color-on-action`
`#f0f2f4`	slot ocupado	`--color-surface-muted`
`#b42318`	errores	`--color-danger`
`#1d4ed8`	anillo de foco	`--color-focus-ring`

`#ffffff` tiene **dos intenciones**: superficie y texto sobre acción → dos tokens.

Manos a la obra · Taller 2

10 MINUTOS

1. **Abre `inicio/componentes.html` con Live Server**
   Se verá sin estilo. Déjalo abierto en una mitad de la pantalla y VS Code en la otra.

2. **T2-1 a T2-4 en `styles/tokens.css` (12 min)**
   Sigue los `TODO T2-n`. Al terminar T2-3 ya cambian los textos y fondos.

3. **T2-5 a T2-7 en `styles/components.css` (12 min)**
   Botones, estados, campos, slots y tarjeta. Cada bloque que completes se ve en la vitrina.

4. **Bonus: tema oscuro (si te sobra tiempo)**
   `[data-theme="dark"]` en `tokens.css` y prueba el botón de la vitrina.

Cómo sabes que terminaste

- `componentes.html` muestra 4 variantes y 4 estados de botón; con Tab ves el anillo de foco
- `Ctrl + F` → `#` en `components.css` devuelve **0 resultados**
- Cambias `--teal-600` por `#6d28d9` y **todo** el producto cambia de marca

styles/tokens.css — capa 1: primitivos (T2-2)

taller-2-design-system/final/styles/tokens.css · [T2-2]

```
:root {
  --teal-600: #0b6e75;        /* marca */
  --teal-700: #095a60;
  --teal-500: #14a3ad;        /* para el tema oscuro */

  /* neutros: 0 = blanco … 950 = casi negro */
  --gray-0:   #ffffff;
  --gray-600: #52606d;
  --gray-900: #1f2933;

  --red-600: #b42318;  --blue-600: #1d4ed8;

  /* espaciado: escala base 4px */
  --space-1: 4px;  --space-2: 8px;  --space-4: 16px;

  /* radios */
  --radius-sm: 8px;  --radius-md: 12px;

  /* tipografía en rem: respeta la preferencia del usuario */
  --font-size-md: 1rem;   --font-size-xl: 1.25rem;
}
```

¿Qué hace cada parte?

1. **Familia + escala**
   `teal-600` dice qué es, no para qué sirve. Los números permiten agregar tonos (650) sin renombrar.

2. **Neutros de 0 a 950**
   La misma convención que Tailwind y Material: el número indica la oscuridad.

3. **Colores de estado**
   Rojo para peligro, azul para el foco. Aún sin significado: eso lo da la capa 2.

4. **Escala de 4 px**
   Toda distancia del producto sale de aquí. Prohibido 13 px.

5. **Radios**
   Dos o tres valores bastan: controles, tarjetas y "píldora" (999 px).

6. **rem para texto**
   Si el usuario configuró letra grande en el navegador, la interfaz la respeta (1.4.4).

styles/tokens.css — capa 2: semánticos y tema oscuro (T2-3)

taller-2-design-system/final/styles/tokens.css · [T2-3]

```
:root {
  --color-bg:                 var(--gray-50);
  --color-surface:            var(--gray-0);
  --color-text:               var(--gray-900);   /* 14.8:1 */
  --color-text-muted:         var(--gray-600);   /*  6.5:1 */
  --color-border:             var(--gray-400);   /*  3.7:1 */
  --color-action-primary:     var(--teal-600);   /*  6.0:1 */
  --color-action-primary-hover: var(--teal-700);
  --color-on-action:          var(--gray-0);
  --color-focus-ring:         var(--blue-600);
}

[data-theme="dark"] {
  --color-bg:                 var(--gray-950);
  --color-surface:            var(--gray-900);
  --color-text:               var(--gray-50);
  --color-action-primary:     var(--teal-500);
  --color-on-action:          var(--gray-950);
}
```

¿Qué hace cada parte?

1. **Nombres por intención**
   `--color-surface` es "la superficie de una tarjeta", sea blanca o gris oscuro.

2. **El contraste documentado**
   Cada par texto/fondo anota su ratio: la regla AA queda visible en el código.

3. **3:1 para bordes**
   El borde no es texto: le basta 3:1 (WCAG 1.4.11).

4. **Texto sobre acción**
   Un token propio porque en el tema oscuro deja de ser blanco.

5. **Tema oscuro = otra capa 2**
   `[data-theme]` en `<html>` cambia los valores; los nombres no cambian.

6. **La marca se aclara en oscuro**
   teal-500 sobre gris oscuro mantiene el contraste; teal-600 sería demasiado oscuro.

Tokens de componente y Button: base y variantes (T2-4, T2-5)

tokens.css (capa 3) + components.css · Button

```
/* tokens.css · capa 3 */
:root { --button-height: 48px;
        --button-radius: var(--radius-sm); }

/* components.css */
.btn {
  --_bg: var(--color-action-primary);       /* tokens locales */
  --_bg-hover: var(--color-action-primary-hover);
  --_fg: var(--color-on-action);
  --_border: transparent;
  min-height: var(--button-height);
  border-radius: var(--button-radius);
  background: var(--_bg);  color: var(--_fg);
  border: 1px solid var(--_border);
}
.btn--secondary { --_bg: var(--color-surface);
  --_fg: var(--color-action-primary); --_border: var(--_fg); }
.btn--danger { --_bg: var(--color-danger);
               --_bg-hover: var(--color-danger-hover); }
```

¿Qué hace cada parte?

1. **Contrato del componente**
   Alto y radio del botón en un solo lugar. 48 px: cómodo al tacto.

2. **Tokens locales (--_)**
   El guion bajo indica "privado del componente". Son los únicos valores que una variante puede cambiar.

3. **La base usa los locales**
   `background: var(--_bg)`: la regla se escribe una sola vez.

4. **Una variante = reasignar**
   `--secondary` solo cambia 3 tokens. No repite padding, fuente ni estados.

5. **Agregar una variante es barato**
   "success" serían 2 líneas. Ese es el patrón que pediremos a la IA.

components.css — los 6 estados del botón (T2-6)

taller-2-design-system/final/styles/components.css · [T2-6]

```
1  .btn:hover:not(:disabled)   { background: var(--_bg-hover); }

2  .btn:active:not(:disabled) { transform: translateY(1px); }

3  :focus-visible { outline: var(--focus-ring-width) solid
                             var(--color-focus-ring); }
4  .btn:disabled {
     background: var(--color-disabled-bg);
     color: var(--color-disabled-text); cursor: not-allowed; }
5  .btn.is-loading { pointer-events: none; cursor: progress; }
   .btn.is-loading::before {
     content: ""; width: 1em; height: 1em; border-radius: 50%;
     border: 2px solid currentColor;
     border-right-color: transparent;
     animation: btn-spin .8s linear infinite; }
6  @media (prefers-reduced-motion: reduce) {
     .btn.is-loading::before { animation-duration: 2.4s; } }
```

¿Qué hace cada parte?

1. **hover usa el token local**
   Por eso funciona igual en primary, secondary y danger. `:not(:disabled)` evita el hover en botones inactivos.

2. **active**
   Un desplazamiento de 1 px da sensación física de "presionado".

3. **Foco global**
   Un solo `:focus-visible` para todo el producto: imposible olvidarlo en un componente nuevo.

4. **disabled**
   Colores propios con contraste legible: deshabilitado no significa invisible.

5. **loading sin HTML extra**
   El spinner es un `::before` con `currentColor`: toma el color del texto de cualquier variante.

6. **Movimiento reducido**
   Más lento, no eliminado: el usuario sigue viendo que algo pasa.

Field con tokens y refactor de reserva.html (T2-7, T2-8)

components.css · Field + final/reserva.html

```
1  .field__control {
     min-height: var(--field-height);
     border: 1px solid var(--field-border);
     border-radius: var(--field-radius);
2  }
3  .field__control[aria-invalid="true"] {
     border: 2px solid var(--color-danger); }
   .field__error { color: var(--color-danger); }

4  <!-- reserva.html: solo clases del design system -->
   <link rel="stylesheet" href="styles/tokens.css">
   <link rel="stylesheet" href="styles/components.css">
5  <label class="field__label" for="nombre">Tu nombre</label>
   <input class="field__control" id="nombre">
   <button type="submit" class="btn btn--block">
     Confirmar reserva</button>
```

¿Qué hace cada parte?

1. **Nombres BEM**
   `field__control`: bloque + elemento. Deja claro a qué componente pertenece cada clase.

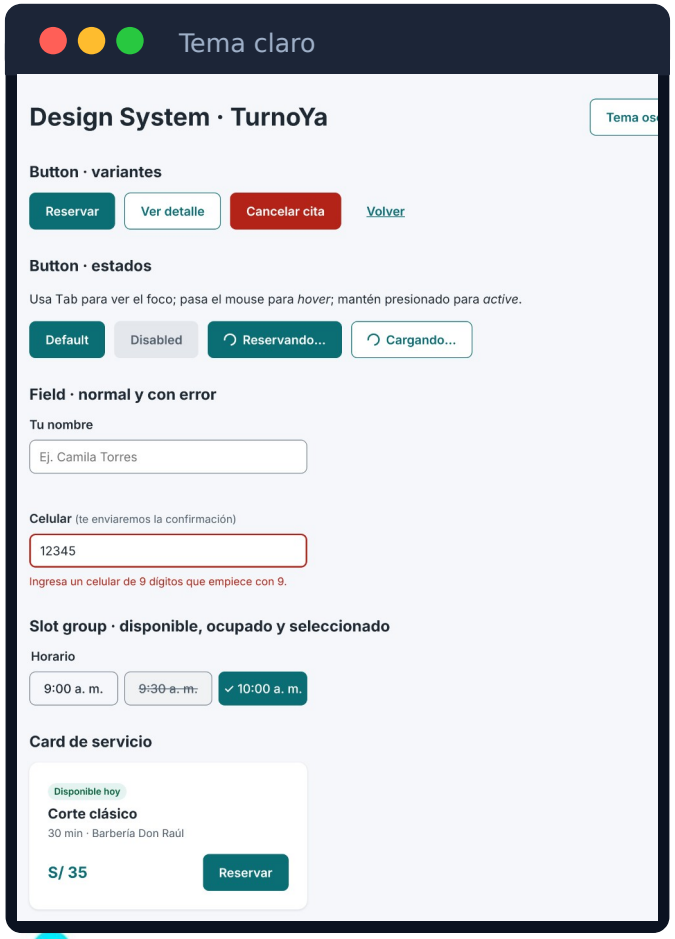
2. **El estilo sigue al atributo ARIA**
   El borde rojo aparece por `aria-invalid`: accesibilidad y estilo nunca se desincronizan.

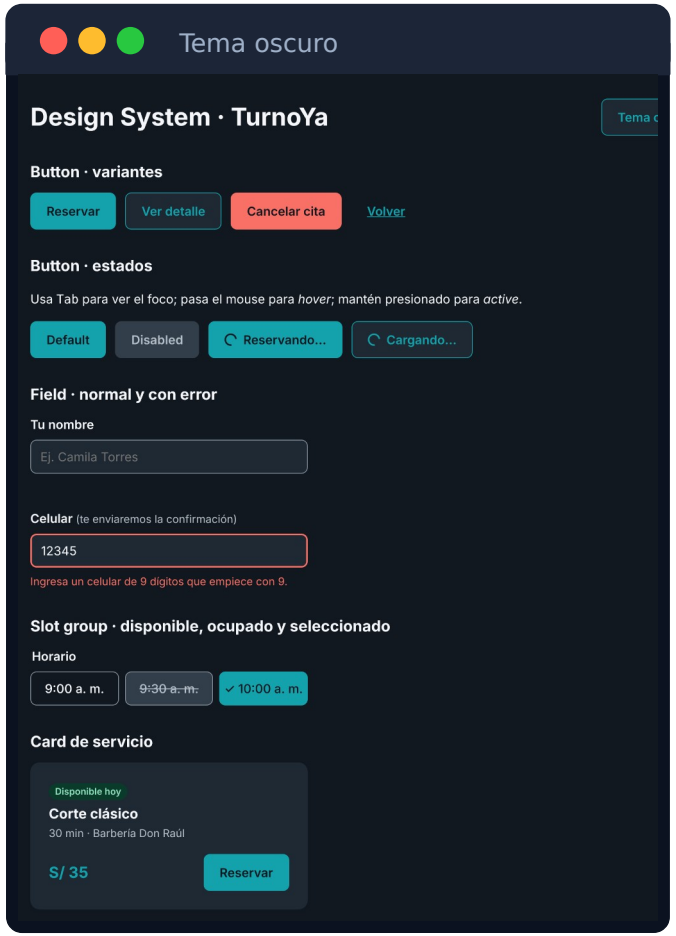
3. **Error con token semántico**
   En el tema oscuro el rojo se aclara solo.

4. **Las hojas de estilo del sistema**
   El `<style>` de la página pasó de ~45 líneas con #hex a 8 líneas de layout.

5. **HTML que solo compone**
   La página no define cómo se ve un botón: usa `.btn`. Es la regla de Atomic Design.

Resultado esperado (Taller 2)





Cómo verificar que funcionó

- 4 variantes y 4 estados visibles
- 0 resultados al buscar `#` en `components.css`
- Cambias `--teal-600` y cambia todo
- El tema oscuro no toca ningún componente

Qué aprendiste en este taller

- Inventariar por intención
- Tokens en 3 capas y quién cambia cada una
- Variantes que reasignan, estados que se escriben una vez
- La página solo compone componentes

06

Responsive & Mobile Design

Una sola base de código para el celular de Camila y la tablet de Don Raúl

¿Qué es el diseño responsive?

Término acuñado por **Ethan Marcotte en 2010**: una sola página que **se adapta al espacio disponible** en lugar de tener una versión por dispositivo. Hoy más de la mitad del tráfico web mundial llega desde celulares, así que el móvil es el caso principal, no el secundario.

Los 3 ingredientes originales (y su versión moderna)

Grillas fluidas

Proporciones en lugar de anchos fijos.

Imágenes flexibles

Nunca más anchas que su contenedor..

Media queries

Reglas que aplican según el ancho.
Hoy: también container queries y unidades fluidas.

Responsive

Una sola base de código, layout **fluido** que se reacomoda de forma continua.

Adaptativo

Varios layouts fijos que se eligen según el dispositivo. Más control, más mantenimiento.

El viewport y las unidades de CSS

La línea que hace posible el responsive

```
<meta name="viewport"
      content="width=device-width, initial-scale=1">
<!-- sin esto el celular simula ~980 px y achica todo -->
```

`width=device-width` usa el ancho real del dispositivo; `initial-scale=1` arranca sin zoom. **Nunca** agregues `user-scalable=no` ni `maximum-scale=1`: bloquean el zoom (WCAG 1.4.4).

`px`	**Absoluta** Bordes, sombras, detalles finos
`rem`	**Relativa al tamaño base del usuario** Tipografía y espaciado
`em`	**Relativa a la fuente del elemento** Padding de un botón que escala con su texto
`%`	**Relativa al contenedor** Anchos fluidos
`vw / vh`	**1 % del viewport** Tipografía fluida dentro de `clamp()`
`dvh`	**Viewport dinámico** Pantallas de alto completo en móvil
`ch`	**Ancho del carácter "0"** Longitud de línea: `max-width: 65ch`
`fr`	**Fracción del espacio libre en Grid** Columnas que reparten el espacio

Flexbox o Grid: una dimensión o dos

No compiten, se complementan. **Flexbox** distribuye elementos en **una dirección** (fila o columna) según su contenido. **Grid** define **filas y columnas a la vez** y coloca los elementos en esa estructura.

Flexbox · header, filtros, botones con ícono

```
.site-header {
  display: flex;
  justify-content: space-between;  /* extremos */
  align-items: center;              /* centrado vertical */
  gap: var(--space-3);
}
```

Grid · catálogo de tarjetas, layouts de página

```
.service-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: var(--space-4);          /* filas y columnas */
}
```

- **Usa Flexbox cuando…**
  el tamaño de cada elemento depende de su contenido: menús, barras de acciones, chips, un ícono junto a un texto.
- **Usa Grid cuando…**
  necesitas alinear en dos ejes o que los elementos respeten una estructura: catálogos, formularios en columnas, el layout de toda la página.

`gap` funciona en los dos

Olvida los `margin-right` en el último elemento: `gap` separa sin agregar espacio en los bordes.

DWC | INSTITUTE

Layout intrínseco

Un layout **intrínseco** deja que el navegador decida según el espacio, sin media queries. Con `auto-fit`, `minmax()` se resuelve la mayor parte del responsive.

Grilla que se acomoda sola

```
.service-grid {
  display: grid; gap: var(--space-4);
  grid-template-columns:
    repeat(auto-fit, minmax(min(100%, 280px), 1fr));
}
```

1. **`auto-fit`**
   crea tantas columnas como quepan; `auto-fill` deja columnas vacías si sobran.

2. **`minmax(280px, 1fr)`**
   cada columna mide al menos 280 px y reparte el sobrante.

3. **`min(100%, 280px)`**
   si el contenedor mide menos de 280 px, usa el 100 %: nunca desborda.



Diseñar para el pulgar: zona de alcance y áreas táctiles

En el estudio de Steven Hoober (2013), el **49 %** de las personas sostenía el celular con una mano y usaba el pulgar. La parte baja y central de la pantalla es la más cómoda; las esquinas superiores, las más difíciles.



Tamaños táctiles de referencia

24 × 24 px	WCAG 2.5.8 (AA) · mínimo
44 × 44 pt	Apple Human Interface Guidelines
48 × 48 dp	Material Design (Android)
44 × 44 px	WCAG 2.5.5 (AAA) · recomendado

DWC | INSTITUTE

Formularios en móvil: el teclado correcto en cada campo

Escribir en un celular es lento. El HTML puede **elegir el teclado, autocompletar y cambiar la tecla Enter** sin una línea de JavaScript.

Campos de la reserva de TurnoYa

```
1 <input type="tel" inputmode="numeric" autocomplete="tel">
2 <input autocomplete="name" autocapitalize="words">
3 <input type="email" autocomplete="email">
4 <input type="date" min="2026-10-26">
5 <input enterkeyhint="send">        <!-- Enter dice "Enviar" -->
```

En iOS, un campo con letra **menor a 16 px** hace zoom automático al enfocarlo. Mantén `font-size: 1rem` en los inputs.

¿Qué hace cada parte?

1. **type="tel" + numeric**
   Teclado numérico grande para el celular.
2. **autocomplete="name"**
   El navegador ofrece el nombre guardado (H7, WCAG 1.3.5).
3. **type="email"**
   Teclado con @ y validación nativa.
4. **type="date"**
   Selector nativo del sistema (Jakob, H5).
5. **enterkeyhint**
   Cambia la etiqueta de la tecla Enter: "Siguiente", "Enviar", "Buscar".

Recursos para seguir profundizando

Recurso	Para qué sirve
w3.org/WAI/WCAG22/quickref	Referencia rápida de WCAG 2.2, filtrable por nivel y tecnología
nngroup.com/articles/ten-usability-heuristics	Las 10 heurísticas explicadas por Nielsen Norman Group
lawsofux.com	Leyes de UX con ejemplos y fuentes originales
webaim.org/resources/contrastchecker	Calculadora de contraste WCAG
developer.mozilla.org · ARIA	Guía de roles y atributos ARIA (MDN)
designtokens.org	Especificación del formato de design tokens del W3C
web.dev/learn/design	Curso gratuito de responsive design de Google

Resumen de la sesión

1	**Principios de UX/UI**	Jerarquía, tipografía, color, espaciado, Gestalt, 10 heurísticas de Nielsen con severidad y 6 leyes de UX.
2	**Accesibilidad WCAG 2.2**	POUR, niveles A/AA/AAA, criterios clave, contraste, ARIA mínimo y cómo probar.
3	**Components & Design Systems**	Atomic design, tokens en 3 capas, nomenclatura, anatomía, variantes y estados de UI.
4	**Responsive & Mobile**	Unidades, Flex y Grid, layout intrínseco, zona del pulgar.
5	**TurnoYa**	Reserva accesible, design system con tema oscuro, catálogo responsive y wireframe.



¿Preguntas?

Gracias por su participación — nos vemos en la próxima sesión